

Средства симуляции ЦП и ОС

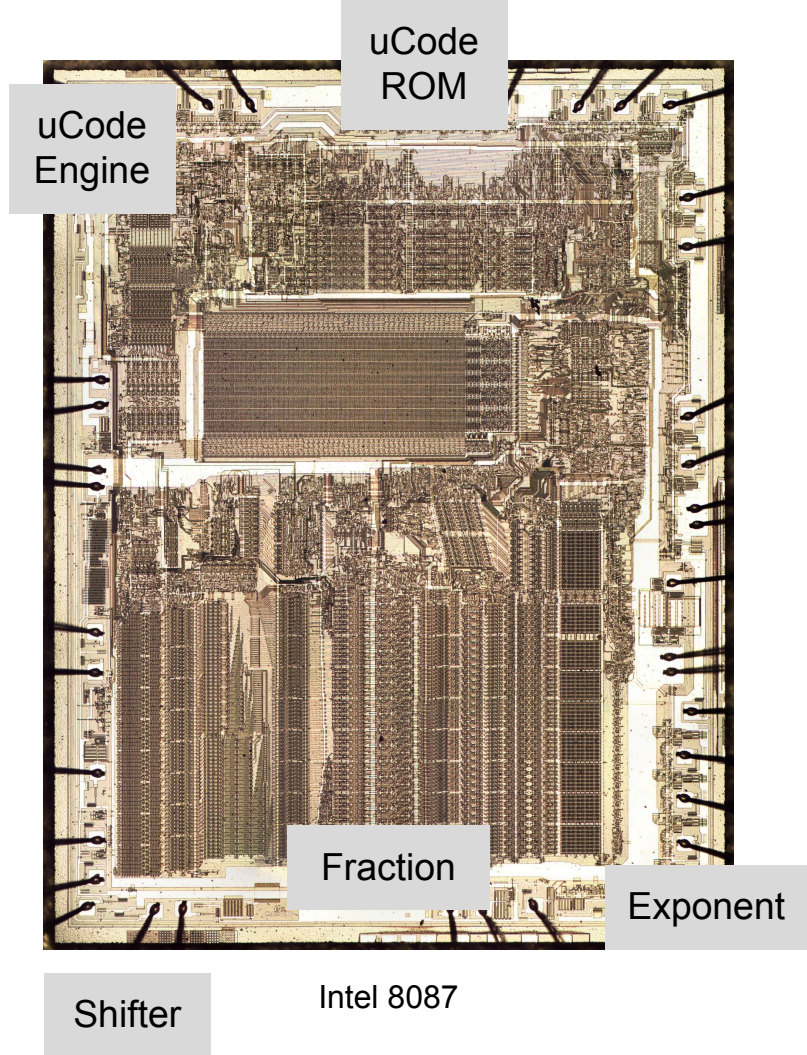
Лекция №19

Державин Андрей
Шурыгин Антон

ИСТОКИ

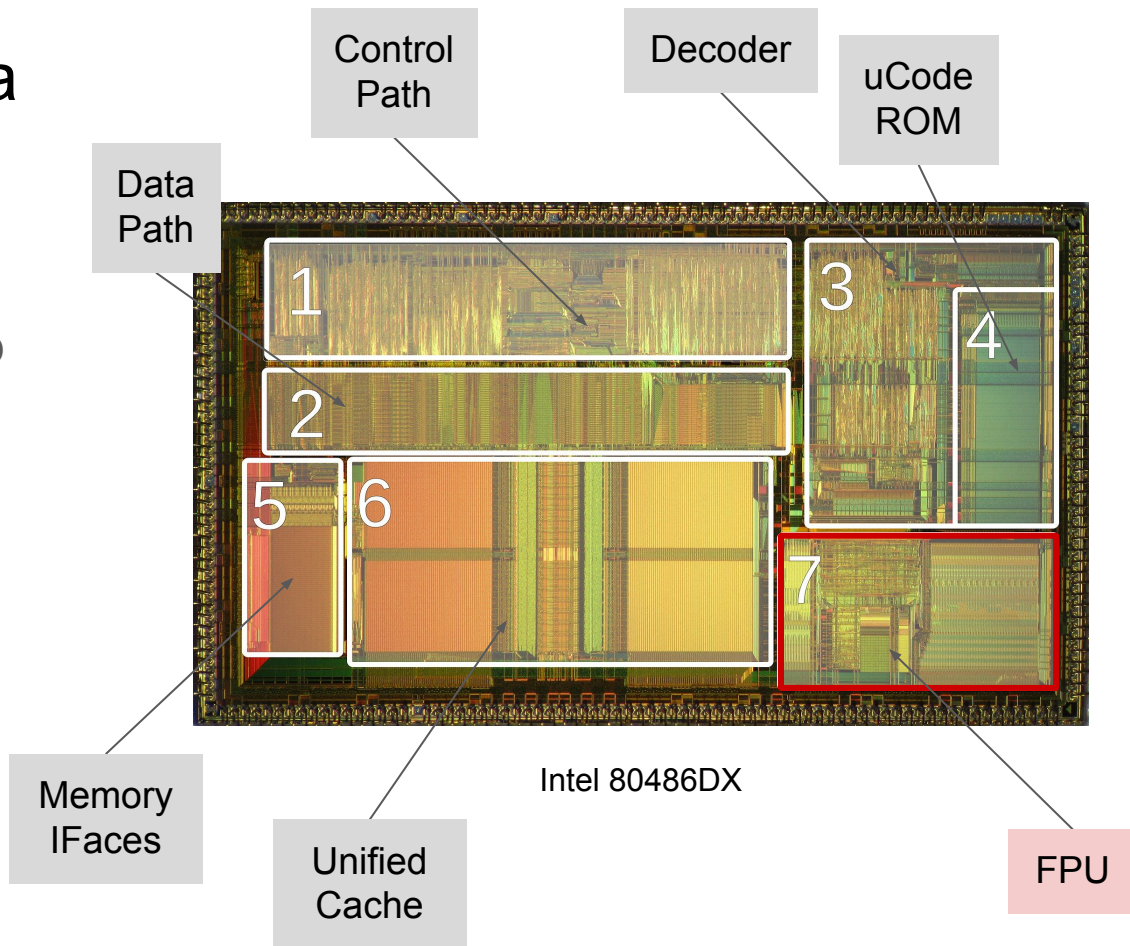
Историческая справка

- Intel 8086 работал вместе с математическим сопроцессором
- Сопроцессор поддерживал набор команд x87 и работал с регистровым стеком
- При покупке ПК с I8086 можно было выбрать комплектацию
 - С сопроцессором – дороже
 - Без сопроцессора – дешевле, вычисление FP через эмуляцию



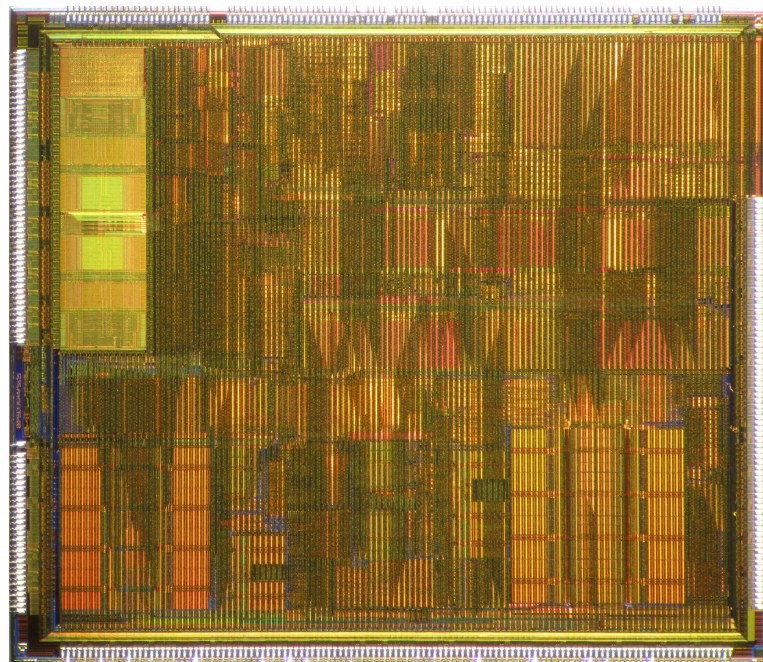
Историческая справка

- В 1989 году в i486 сопроцессор был интегрирован в процессор как полноценное устройство – FPU
- Возникли новые 8 80-битных регистров для хранения FP чисел
- 80486SX – маркетинговое упрощение



Историческая справка

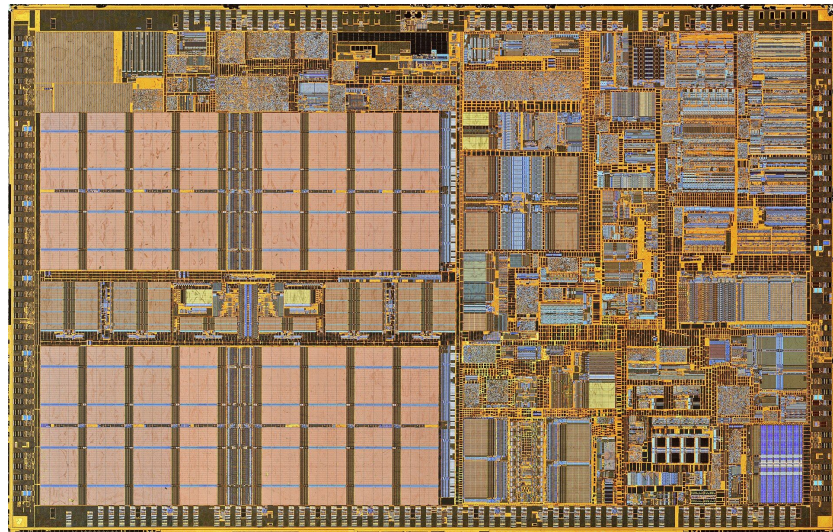
- В 1997 году выходит Pentium MMX
 - 57 MMX-команд для ускорения обработки графики и звука
- Были переиспользованы уже существующие 80-битные регистры FPU как 64-битные MMX
- В итоге – наложению архитектурных состояний.
 - Нельзя было одновременно использовать инструкции FPU и MMX



Pentium MMX (p55c) die

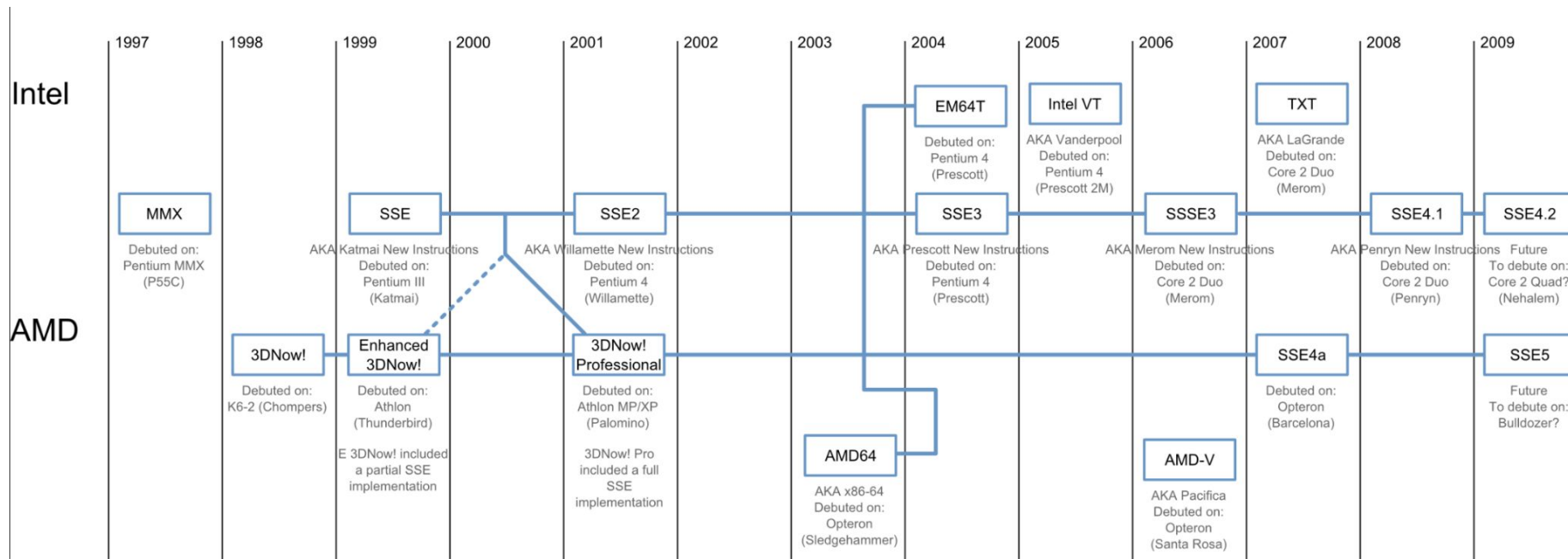
Историческая справка

- В 1999 году Intel представила Streaming SIMD Extension (SSE)
 - Независимый регистровый файл — восемь 128-битных регистров — XMM0 .. XMM7
 - Теперь SIMD-инструкции могли работать параллельно с FPU



Pentium III die

Историческая справка

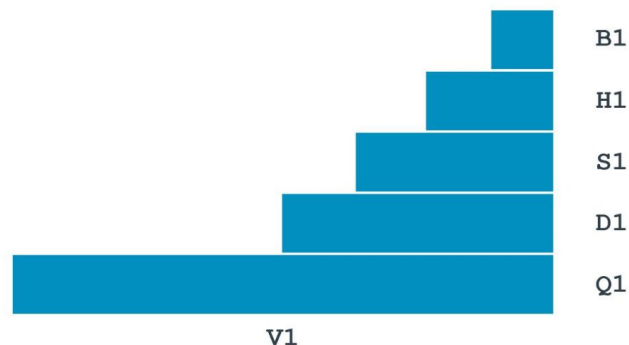
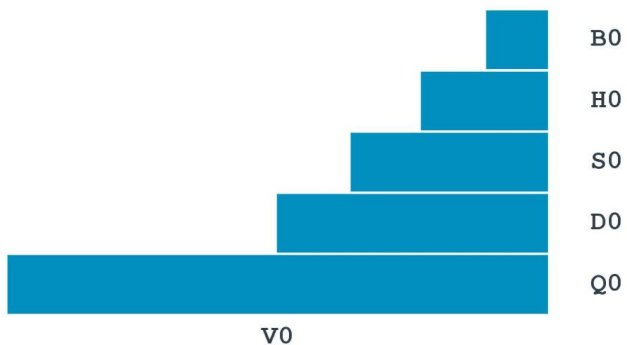


Хронология.
Правда только до 2009 года

AArch64

AArch64

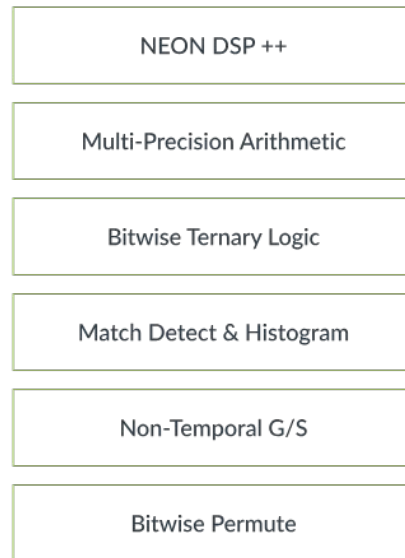
- Архитекторы заложили SIMD&FP так, что:
 - Регистровый файл — тридцать два 128-битных регистров – V0 .. V31
 - Одновременно используется и для SIMD (NEON), и для скалярных операций с плавающей точкой (VFP)



Векторное расширение

- Два типа векторной обработки
 - Advanced SIMD, также известный как NEON.
 - Scalable Vector Extension (SVE и SVE2).
- SVE был представлен в Armv8-A и оптимизирован для высокопроизводительных вычислений

SVE2



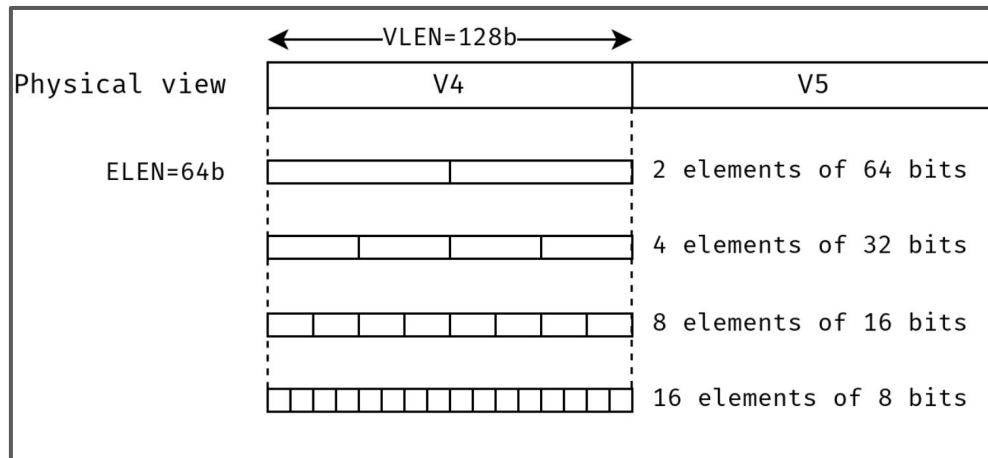
SVE



RISC-V Vector Extension

RVV State

- 32 векторных регистра **v0-v31**
- VLEN - максимальная длина **одного** вектора (в битах)
 - Задаётся реализацией
 - Степень двойки
- Векторы делятся на **элементы**
 - ELEN - максимальная длина элемента (в битах)
 - Задаётся реализацией
 - Степень двойки
 - $8 \leq \text{ELEN} \leq \text{VLEN}$



RVV

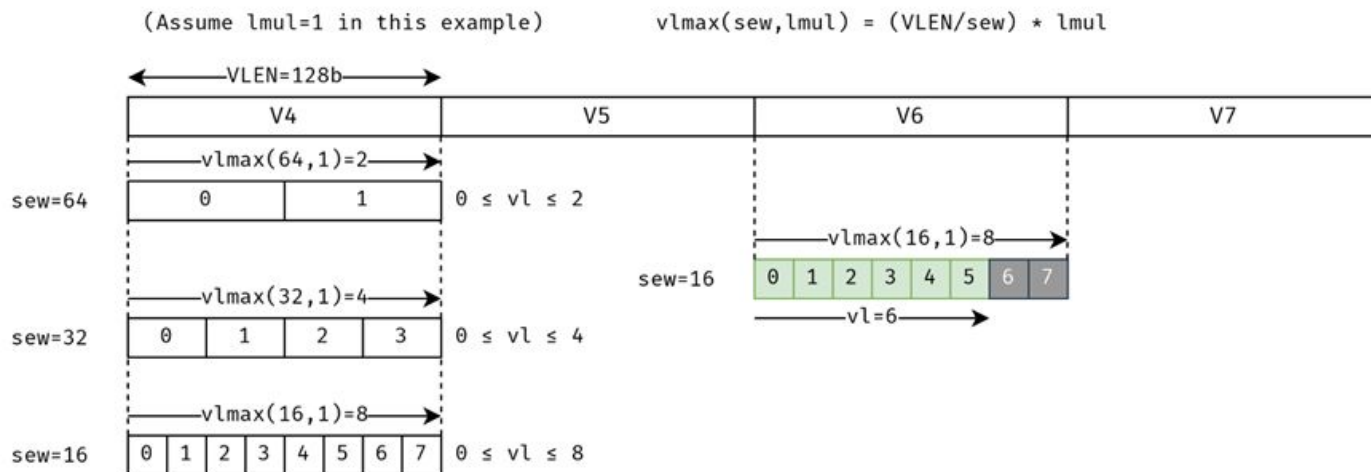


- RISC-V Vector extension
- Масштабируемость
- Независимость от длины вектора
 - Её даже можно менять!
- Гибкий дизайн

RVV: Как настроить?

- **vtype** описывает раскладку вектора
 - **sew**: Размер элемента вектора в битах
 - $(8 \leq \text{sew} \leq \text{ELEN})$
 - **lmul**: группировка регистров
 - $\text{lmul} = 2^k, k = -3 \dots 3$
- **vl** - число элементов в векторе
 - $0 \leq \text{vl} \leq \text{vlmax}(\text{sew}, \text{lmul})$
 - $\text{vlmax}(\text{sew}, \text{lmul}) = (\text{VLEN} / \text{sew}) \times \text{lmul}$

RVV: Как настроить?



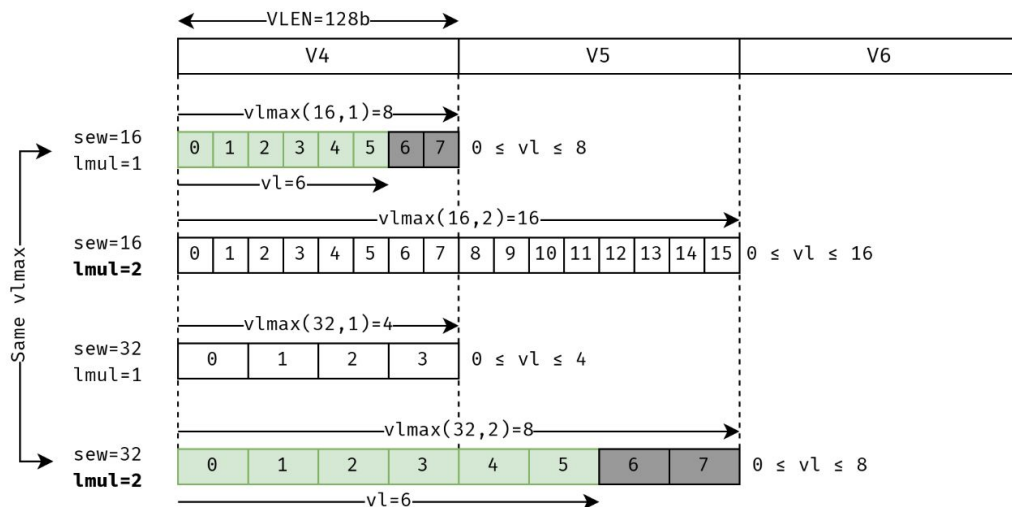
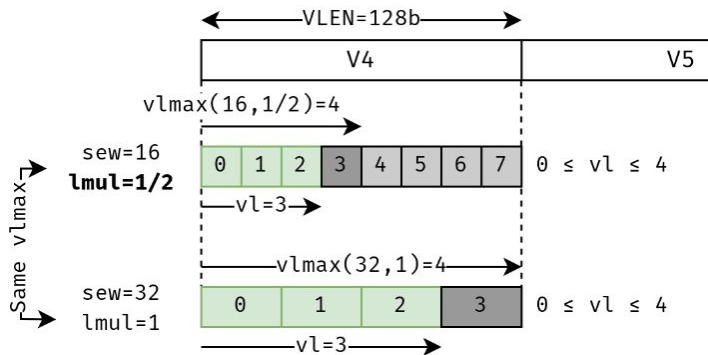
Зачем это все?

- Пока что всё выглядит сложно
- Может быть на то есть причины?
- Очевидно, что $vl \sim 1/sew$
- В алгоритмах иногда нужно работать с разными sew
 - Но при этом одинаковым vl
- Что можно сделать?
 - Не использовать весь регистр для малых sew
 - **Группировать** регистры для больших sew



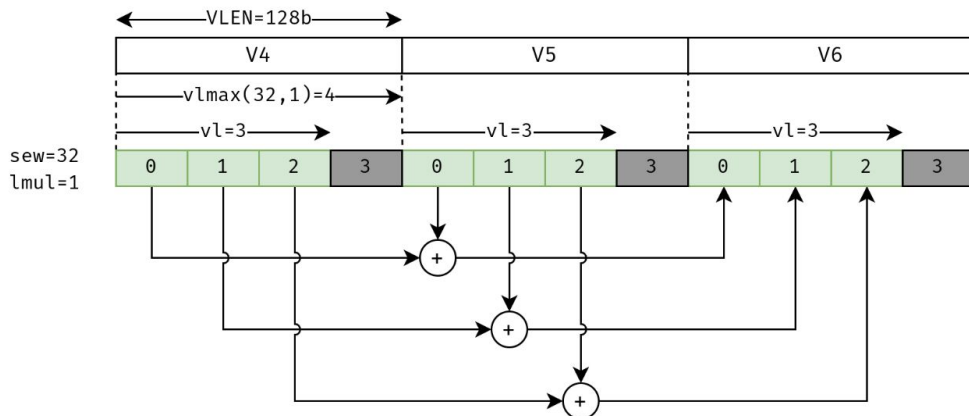
LMUL?

- $lmul = 1$ - доступны все элементы вектора
- $lmul < 1$ - доступна часть элементов вектора ($\frac{1}{2}$, $\frac{1}{4}$, $\frac{1}{8}$)
- $lmul > 1$ - операция производится над **группой** из $lmul$ регистров
 - Группа задаётся номером младшего регистра
 - 4 регистра при $lmul=8$
 - $v0, v8, v16, v24$



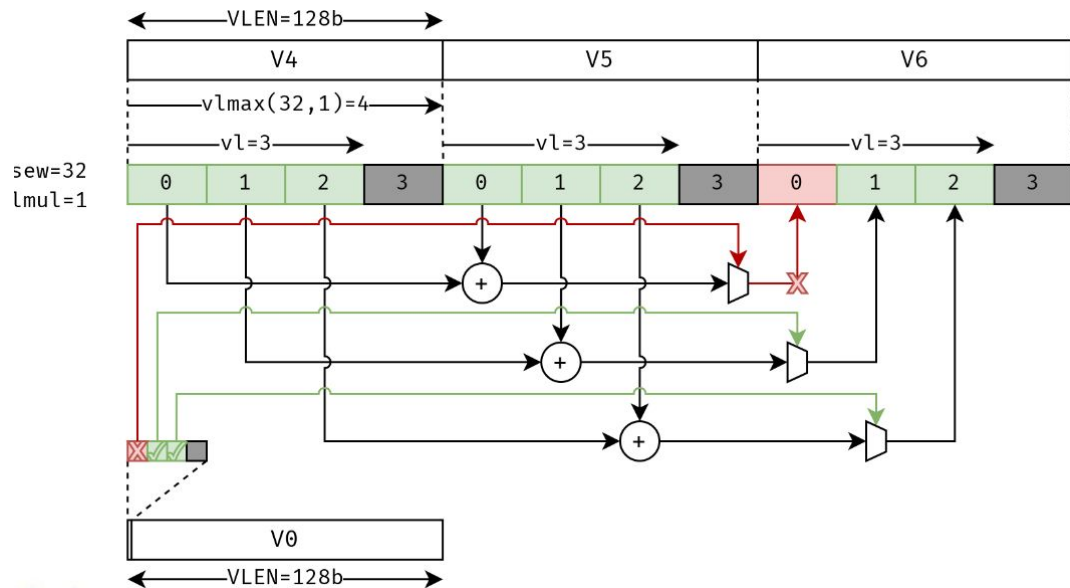
RVV с хвостиком

- Что делать с хвостом ($vl < vlmax$)?
- RVV предлагает два поведения:
 - **tail undisturbed** - хвост не меняется
 - **tail agnostic** - хвост либо не меняется, либо выставляется в 1
- С **undisturbed** понятно, зачем нужен **agnostic**?



RVV предикаты

- Нет отдельных регистров для масок
- Используются v регистры
- Непосредственно для маскирования используется v0



RVV маски

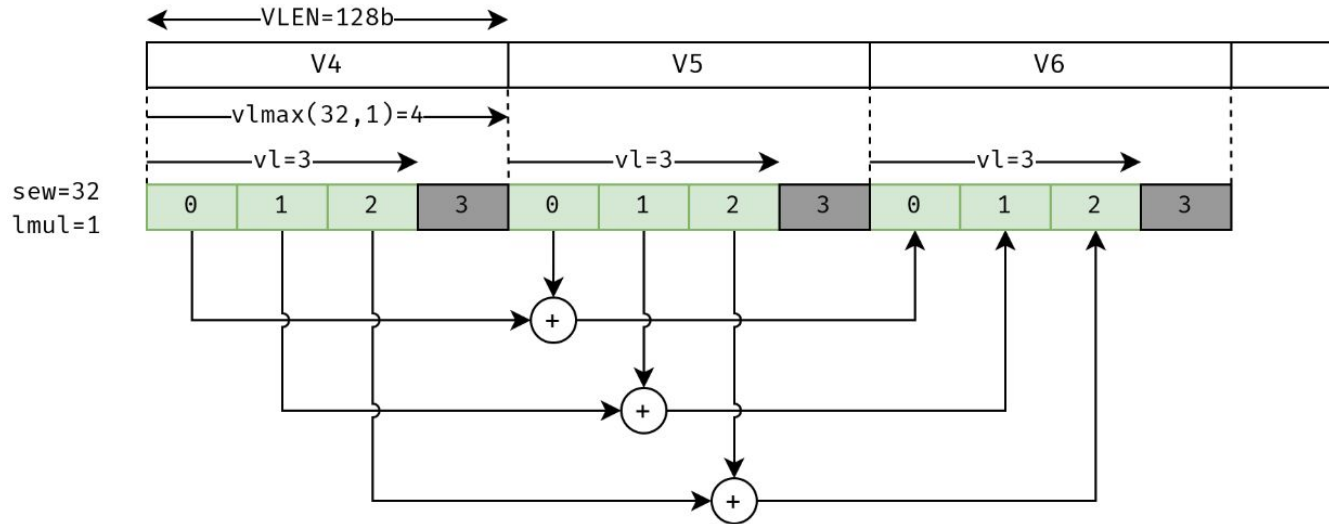
- С поведением активного элемента все понятно
- А что с неактивным?
- Опять 2 политики
 - **mask undisturbed** - элемент не меняется
 - **mask agnostic** - либо меняется, либо выставляется в 1

vsetvli

- Обычные инструкции не могут менять vl и vtype
 - vsetvl, vsetvli, vsetivli
 - vle*ff
- vsetvli rd, rs, eN, mX, tP, mP
 - rs - application vector length (AVL) == хочу столько элементов
 - rd выставляется в итоговую vl == будешь обрабатывать сколько скажу
 - N - sew (8, 16, 32, 64...)
 - X - lmul > 1, fY lmul < 1
 - P - политика для хвостов и масок (A или U)
- vsetvli rd, x0, eN, mX, tP, mP
 - Выставляет rd = vl = vlmax
- vsetvli x0, x0, eN, mX, tP, mP
 - Изменяет только vtype. (Валидна если vlmax не изменилась)

vsetvli

```
vsetivli x10, 3, e32,m1,ta,ma # vl ← 3, sew ← 32, lmul ← 1  
vadd.vv v6, v4, v5
```



Если запросы велики?

- Спецификация задаёт
 - $vl = \min(vlmax, AVL)$
- Если $vlmax < AVL < 2 * vlmax$, то можно схитрить:
 - $\text{ceil}(AVL / 2) < vl < vlmax$
- VLEN 128
- $vlmax(\text{sew}=16, \text{lmul}=1) = 8$
 - $vsetivli\ x0, 9, e16, m1, ta, ma$
 - $5 \leq vl \leq 8$

RVV memcpy

```
# void *memcpy(void* dest, const void* src, size_t n)
```

```
# a0=dest, a1=src, a2=n
```

```
mv a3, a0 # Copy destination
```

```
loop:
```

```
vsetvli t0, a2, e8, m8, ta, ma # Vectors of 8b
```

```
vle8.v v0, (a1) # Load bytes
```

```
add a1, a1, t0 # Bump pointer
```

```
sub a2, a2, t0 # Decrement count
```

```
vse8.v v0, (a3) # Store bytes
```

```
add a3, a3, t0 # Bump pointer
```

```
bnez a2, loop
```

```
ret
```

[riscv-v-spec/example/memcpy.s](https://riscv-v-spec.github.io/example/memcpy.s) at master

RISC-V FP Extension

RISCV FP State

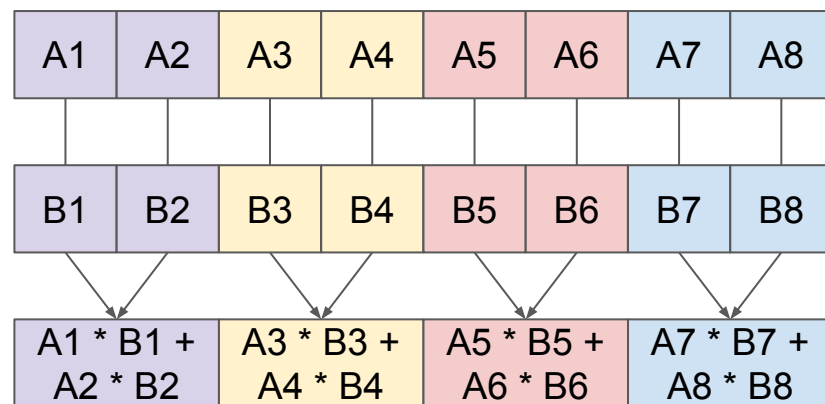
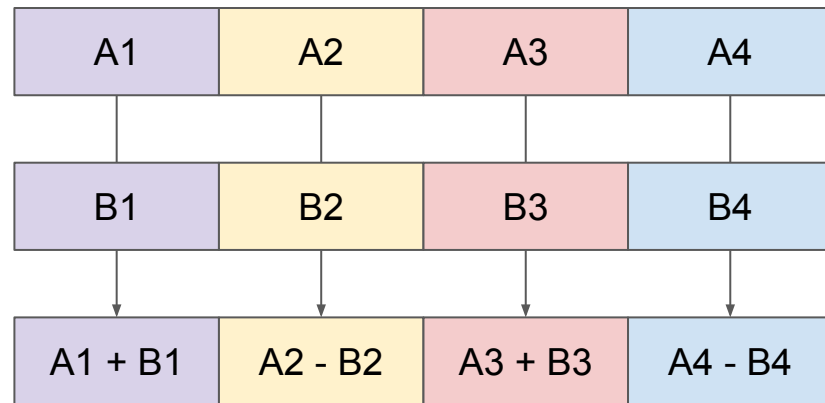
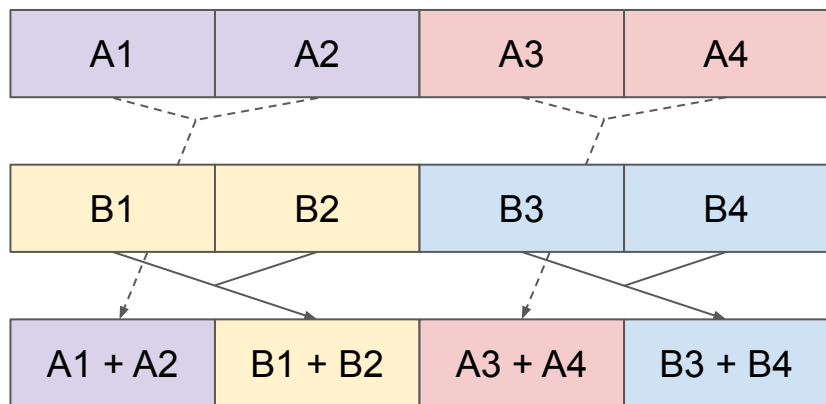
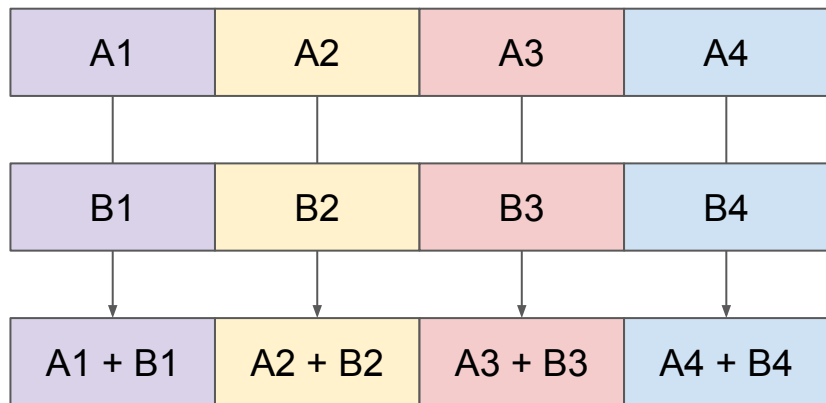
- 32 регистра f0-f31
- FLEN - ширина регистров (FLEN=32 для RISCV-F)
- F, D, Q расширения
- **Zfh** -half-precision
- Z*inx - без FPU, вычисления на x регистрах
- Системный регистр fcsr
 - Rounding mode
 - Exception flags

Векторизация

Векторизация

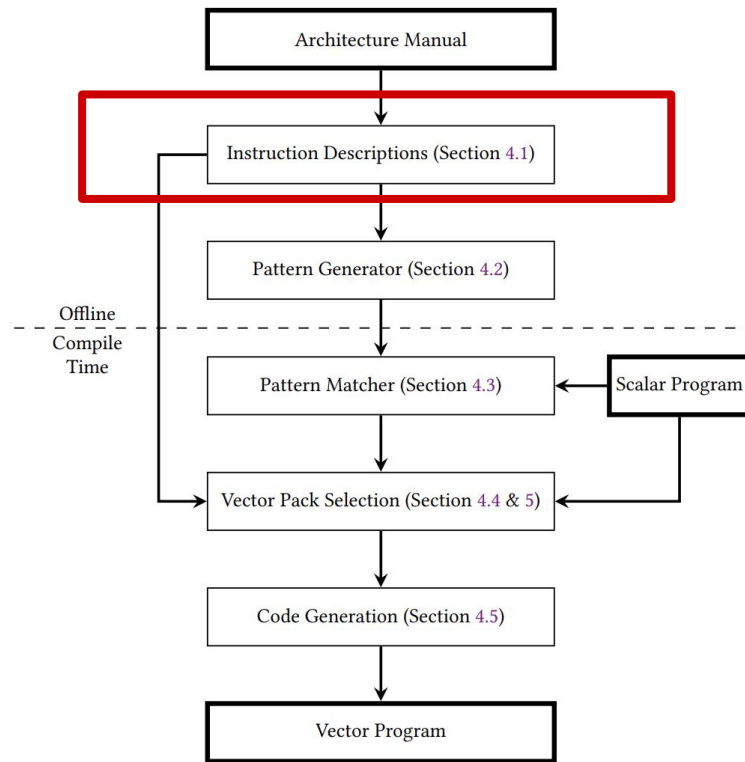
- Классический SIMD: инструкция выполняет одну и ту же операцию над всеми дорожками (lane) независимо
- В реальных ISA существует:
 - SIMMOD (Single Instruction Multiple Operations Multiple Data)
 - Горизонтальные операции
 - Инструкции с внутренними редукциями
- Классическая автовекторизация не умеет работать с не-SIMD инструкциями

Векторизация



VeGen

- Вводится Lane Level Parallelism (LLP) — обобщение SLP, разрешающее:
 - Неизоморфные операции на разных дорожках
 - Статическую cross-lane коммуникацию
- VeGen — это генератор векторизатора, который из семантического описания инструкций (из Intel Intrinsics Guide) порождает:
 - Pattern matchers
 - Lane-binding функции



Рабочий процесс VeGen

Литература

1. [VeGen: A Vectorizer Generator for SIMD and Beyond](#)
2. [Ratified Specifications - RISC-V International](#)
3. [Introduction to the RISC-V Vector Extension](#)